

SQL 注入漏洞报告（Lab10：带时间延迟的盲注）

1. 漏洞名称

SQL Injection（SQL 注入） — 时间盲注（Time-based Blind SQL Injection）

2. 漏洞等级

高危（High）

3. 漏洞描述

该网站使用 TrackingId Cookie 进行跟踪分析，并将该值拼接入后端 SQL 查询。虽然页面不返回任何查询结果，也不会因为 SQL 执行错误而返回不同的响应状态，但由于后端数据库查询是**同步阻塞**的，攻击者可以利用数据库特定的时间延迟函数（如 pg_sleep）触发明显的响应延迟，从而确认注入点的存在。

4. 漏洞位置

请求方式：GET /（首页）

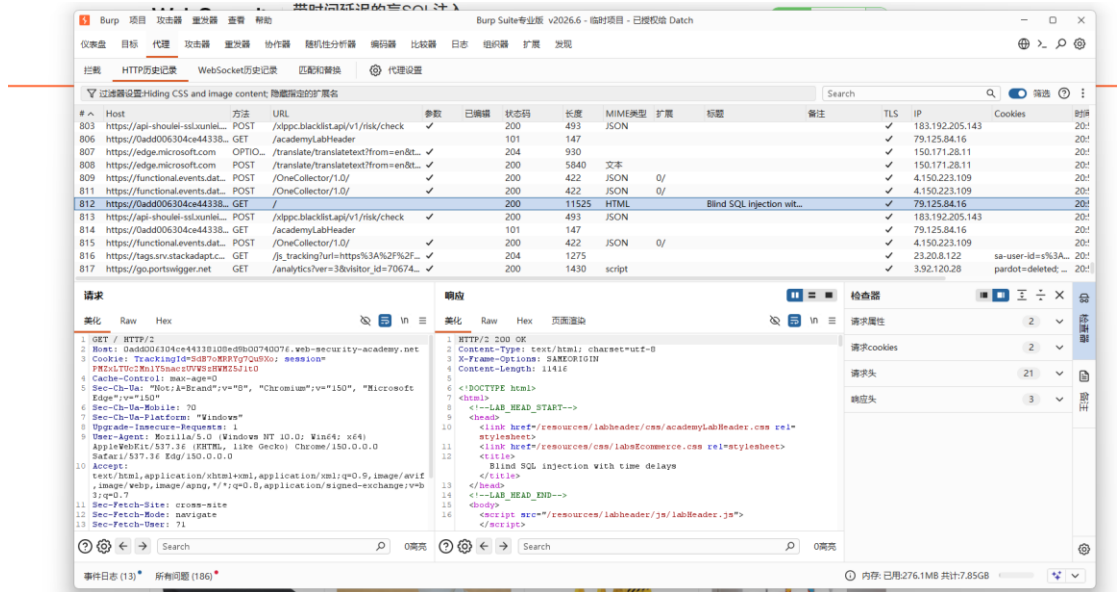
漏洞参数：Cookie: TrackingId=...

5. 漏洞复现过程

第一步：抓包与确认注入点

访问实验首页，使用 Burp Suite 拦截包含 TrackingId 的 GET 请求，并将其发送至 Repeater 模块进行构造。

图 1：Burp Suite HTTP History 抓包记录



说明：从 HTTP History 中找到目标请求，确认 Cookie 参数位置。

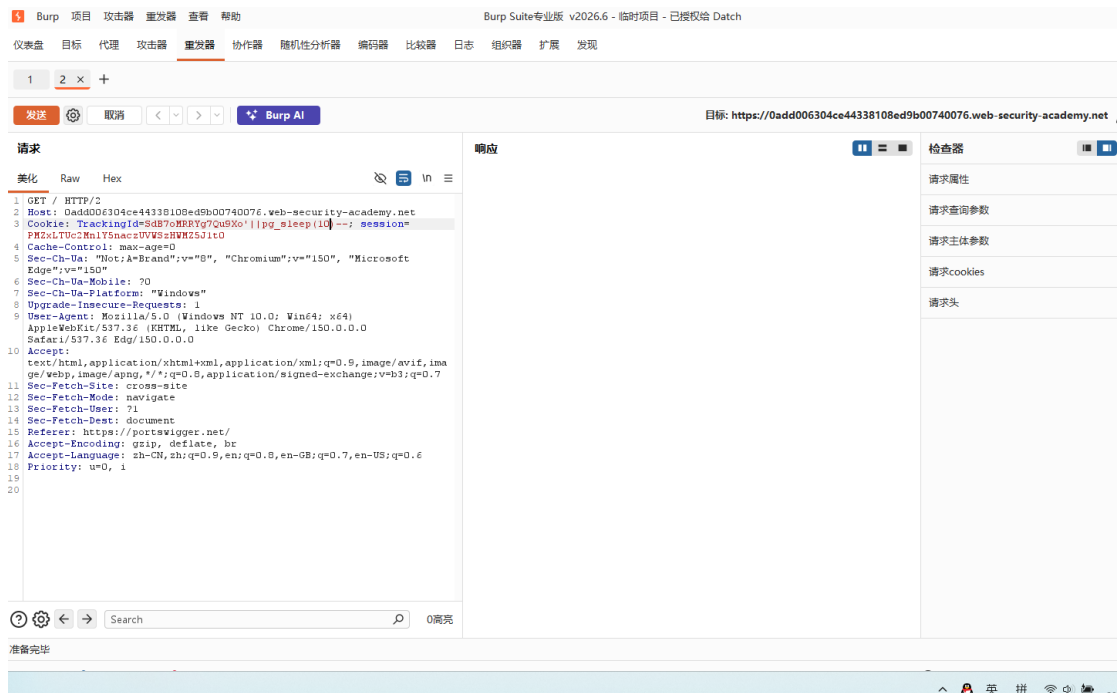
第二步：构造延时 Payload

由于无法通过页面“真假”来判断注入，必须构造特定的延时语句。根据提示，后端数据库为 PostgreSQL，使用 pg_sleep() 函数。

将 TrackingId 的值修改为：

TrackingId=x'||pg_sleep(10)--

图 2：Burp Repeater 构造并发送延时 Payload



说明：通过字符串拼接符 || 将恶意函数拼接到原始 SQL 中，注释符 -- 用于消除后续

逻辑，确保语法合法。

第三步：响应延迟验证

点击 Burp Repeater 的“发送”按钮。

观察 Burp 界面右下角的计时器（或肉眼感受响应时间），发现响应时间出现明显停顿，约 10 秒后才收到服务器的 HTTP 200 响应。

这一明显的延迟，确凿证明 SQL 语句中的 pg_sleep(10) 被后端数据库成功解析并执行。

第四步：漏洞验证完成

成功触发 10 秒延迟后，页面即提示实验完成。

图 3：实验完成与漏洞验证成功



说明：成功触发时间盲注，靶场服务器确认漏洞攻击验证完成。

6. 漏洞原理

原始 SQL 逻辑为：SELECT TrackingId FROM tracking_table WHERE TrackingId = 'x'
经过 Payload 拼接后，实际执行的 SQL 语句变为：

sql

SELECT TrackingId FROM tracking_table WHERE TrackingId = 'x'||pg_sleep(10)--'

- ||: PostgreSQL 中的字符串连接符，将用户输入与原 SQL 拼接。
- pg_sleep(10): PostgreSQL 内置的延时函数，使当前数据库会话挂起 10 秒。
- --: SQL 注释符，注释掉了原有 SQL 语句结尾的 '。

7. 漏洞影响

即使网站做好了布尔盲注的防御（不展示 Welcome back 等差别化信息），攻击者依然可以通过**测量服务器响应时间**来提取数据。

如果这里不是 `pg_sleep(10)`，而是 `pg_sleep(CASE WHEN (SELECT password FROM users...)= 'a' THEN 10 ELSE 0 END)`，攻击者就能通过“是否延迟 10 秒”，逐字把管理员密码盲猜出来。

8. 修复建议

- ① **使用参数化查询（预编译）**：从根本上阻断恶意 SQL 语句注入。
- ② **设置全局请求超时限制**：在 Web 服务器或应用层面，强制所有数据库请求在 3~5 秒内超时。即便攻击者注入了延时函数，服务器也会主动断开连接，使其无法通过测量时间来判断真假。
- ③ **数据库连接池隔离**：确保 Web 应用的数据库连接不会因单次恶意查询而阻塞其他正常用户的请求。

9. 工具

- **Burp Suite**（HTTP History 抓包、Repeater 请求重放）
- **PortSwigger Web Security Academy**（靶场平台）